



LINGUAGEM DE PROGRAMAÇÃO

AULA 5



Prof. Wellington Rodrigo Monteiro



CONVERSA INICIAL

Olá, aluno(a)! Nas nossas primeiras interações com o desenvolvimento de *software*, fomos aprendendo operações básicas e estruturas simples de código para compreendermos como elas funcionariam na prática: isso vai desde a leitura e a escrita de variáveis simples (como *input* e *print* em Python), passando pelas operações matemáticas básicas (como detectar se um número é par ou ímpar), até as estruturas de repetição de código (como o *for* e o *while*). Contudo, é improvável que tenhamos parado para pensar na forma como essas técnicas funcionam – isto é, acabamos aprendendo como implementá-las sem nos preocupar muito em como alguém as desenvolveu.

Considerando isto, em algumas aplicações mais complexas, é comum precisarmos implementar algumas dessas estruturas manualmente. Peguemos o exemplo de uma grande empresa com um algoritmo responsável por fazer pedidos de compra de matéria-prima para os fornecedores, a qual possui vários fornecedores e não pode privilegiar um ou outro: ela precisa pedir de forma equilibrada. Agora, como essa empresa poderia criar um algoritmo que selecionasse de forma aleatória alguns fornecedores com base em uma lista maior? Ou, ainda, como poderia gerar algumas combinações de fornecedores para uma análise posterior?

É nesse exemplo de cenário que focaremos esta aula: temos como interesse observar não somente estruturas de iteração (sem o *n*), mas também combinações, permutações (são coisas diferentes) e sobrecarga de operadores. Vamos lá?

TEMA 1 – ITERADORES

As iterações (e não interações, com *n*) são as repetições de algo. Imagine uma lista: se lemos valor por valor nesta lista, dizemos que estamos a percorrendo/executando um *loop* nela/iterando por ela.

É provável que, até este momento, você já utilizou iterações pelos laços de repetição como *for*, *while* ou *while True*, e não há problema algum nisso: você já aplicou conceitos de iterações na prática. Por outro lado, é interessante que saiba que existem formas mais avançadas de trabalharmos com iteradores.



Note que esses exemplos vão além do que vimos até o momento: é necessário, portanto, que saibamos **como colocá-los em prática**. E é por esse motivo que estamos aqui. Vamos lá?

Basicamente, todos os tipos de dados em Python que empregam sequências podem fazer uso dos iteradores, isso inclui *strings*, listas, dicionários, tuplas e afins. Outros tipos de dados que são implementados por outras bibliotecas, como o array do NumPy ou o DataFrame do pandas, também podem ser iterados.

Muito importante: evite modificar uma sequência (isto é, alterar ou remover itens de uma lista, um DataFrame ou qualquer outra variável que esteja sendo utilizada) durante uma iteração.

1.1 Exemplos

Vamos trabalhar alguns exemplos para demonstrar como os iteradores funcionam com diferentes tipos de dados. Começaremos com uma *string* contendo a seguinte frase: *Estamos em testes*.

```
frase = 'Estamos em testes.'

for valor in frase:
    print(f'Processou: {valor}')
```

```
Processou: E
Processou: s
Processou: t
Processou: a
Processou: m
Processou: o
Processou: s
Processou:
Processou: e
Processou: m
Processou:
Processou: t
Processou: e
Processou: s
Processou: t
Processou: e
Processou: s
Processou: .
```

Veja que o *for* divide a *string* e processa um por vez. Isso acontece porque, em Python, uma *string* pode ser entendida como uma lista de caracteres. Também poderíamos dividir de outras formas a *string*: veja o exemplo a seguir, no qual dividimos a *string* toda vez que encontrarmos um espaço:



```
for valor in frase.split(' '):  
    print(f'Processou: {valor}')
```

```
Processou: Estamos  
Processou: em  
Processou: testes.
```

Agora, vamos para outro exemplo utilizando o que vimos anteriormente: mais especificamente, criamos um *pickle* contendo os nomes das equipes que competiram no campeonato da primeira divisão de futebol da Mongólia.

```
import pickle  
  
# salvando o array em um arquivo pickle  
# note que não estamos mais salvando o texto, mas uma variável  
lista_equipas = pickle.load(open(path + 'lista_equipas.pkl', 'rb'))  
lista_equipas
```

```
array(['Athletic 220', 'Falcons', 'Ulaanbaatar', 'Khaan Khuns - Erchim',  
      'Deren', 'Ulaanbaatar City', 'Khangarid', 'Khoromkhon',  
      'BCH Lions', 'UB Mazaalaynuud'], dtype=object)
```

Com base no exemplo anterior, utilizaremos novamente o *for* para percorrer item a item seguindo a ordem dos elementos do *array* do NumPy:

```
for valor in lista_equipas:  
    print(f'Processou: {valor}')
```

```
Processou: Athletic 220  
Processou: Falcons  
Processou: Ulaanbaatar  
Processou: Khaan Khuns - Erchim  
Processou: Deren  
Processou: Ulaanbaatar City  
Processou: Khangarid  
Processou: Khoromkhon  
Processou: BCH Lions  
Processou: UB Mazaalaynuud
```

Isso também se aplica a DataFrames: vamos recarregar a base completa do campeonato de futebol que vimos anteriormente.

```
# dataset do campeonato de futebol da Mongólia  
df_fut = pd.read_csv(path + 'dataset_mongolia.tsv', sep='\t')  
df_fut
```



Posição	Equipe	Partidas	Vitórias	Empates	Derrotas	Gols Pró	Gols Contra	Saldo de Gols	Pontos
1	Athletic 220	9	7	0	2	24	8	16	21
2	Falcons	9	6	1	2	21	10	11	19
3	Ulaanbaatar	9	5	3	1	17	5	12	18
4	Khaan Khuns - Erchim	9	5	2	2	16	6	10	17
5	Deren	9	5	1	3	17	10	7	16
6	Ulaanbaatar City	9	4	2	3	16	13	3	14
7	Khangarid	9	3	1	5	17	27	-10	10
8	Khoromkhon	9	2	1	6	11	25	-14	7
9	BCH Lions	9	2	0	7	14	29	-15	6
10	UB Mazaalaynuud	9	0	1	8	2	22	-20	1

Agora, vamos usar o `for` igual a como fizemos anteriormente:

```
for linha in df_fut:
    print(f'Processou: {linha}')
```

```
Processou: Posição
Processou: Equipe
Processou: Partidas
Processou: Vitórias
Processou: Empates
Processou: Derrotas
Processou: Gols Pró
Processou: Gols Contra
Processou: Saldo de Gols
Processou: Pontos
```

Percebeu algo estranho? Ele não percorreu linha a linha, mas, sim, percorreu o nome das colunas, somente. Isso pode variar dependendo da biblioteca e do tipo de dado, e é por isso que ler a documentação das bibliotecas pode ser uma boa opção para nós. No caso, a documentação do pandas¹ sugere que utilizemos o `iterrows`² para esses cenários. Observe como fica o resultado

¹ Disponível em: <https://pandas.pydata.org/docs/user_guide/basics.html#iteration>. Acesso em: 31 jan. 2022.

² Disponível em: <<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iterrows.html#pandas.DataFrame.iterrows>>. Acesso em: 31 jan. 2022.



do *for* com o *iterrows*. Para fins de visualização, mostraremos somente as duas primeiras linhas (por isso o *head(2)*):

```
# mostrando somente as duas primeiras linhas
for indice, linha in df_fut.head(2).iterrows():
    print(f'Índice: {indice}')
    print(f'Processou: {linha}')
```

```
Índice: 0
Processou: Posição 1
Equipe Athletic 220
Partidas 9
Vitórias 7
Empates 0
Derrotas 2
Gols Pró 24
Gols Contra 8
Saldo de Gols 16
Pontos 21
Name: 0, dtype: object
Índice: 1
Processou: Posição 2
Equipe Falcons
Partidas 9
Vitórias 6
Empates 1
Derrotas 2
Gols Pró 21
Gols Contra 10
Saldo de Gols 11
Pontos 19
```

O *iterrows* faz a iteração pelas linhas do DataFrame e nos fornece dois valores: o índice daquela linha e, depois, os valores nela contidos. Note que os dados em amarelo se referem exatamente à primeira linha do DataFrame e, em azul, à segunda linha. Temos como resultado os nomes das colunas e os seus respectivos valores.

Ao fazermos iterações pelas linhas dos DataFrames, também poderíamos fazer uma nova iteração pelos valores de cada linha – ou seja, iterações dentro de iterações. Observe:

```
# obtendo as linhas do DataFrame
for indice, linha in df_fut.head(2).iterrows():
    # para cada linha, obtemos os valores para cada coluna
    for valor in linha:
        print(f'Processou: {valor}')
```



```
Processou: 1
Processou: Athletic 220
Processou: 9
Processou: 7
Processou: 0
Processou: 2
Processou: 24
Processou: 8
Processou: 16
Processou: 21
Processou: 2
Processou: Falcons
Processou: 9
Processou: 6
Processou: 1
Processou: 2
Processou: 21
Processou: 10
Processou: 11
Processou: 19
```

Veja que todos os exemplos executam iterações até um certo limite: no caso da *string* e do *array*, por exemplo, o *for* executou até o final dos itens. No caso das linhas do *DataFrame*, mostramos todos os valores das duas primeiras linhas porque fomos nós que colocamos esse limite de duas linhas.

Contudo, vale lembrar que o Python possui também funções³ com iteradores infinitos com as funções *count*⁴, *cycle*⁵ e *repeat*⁶. O *count* fará uma contagem de forma infinita até que algo o pare: precisamos informar necessariamente um valor inicial (no exemplo a seguir, 20) e, opcionalmente, um valor que informe qual deve ser a soma para determinar o próximo número (se nada for informado, este valor será 1; no exemplo a seguir, informamos 2, ou seja, será um contador de 2 em 2). Se nada for feito, o contador continuará gerando números tendendo ao infinito. Somente os primeiros cinco números serão mostrados na saída a seguir para fins ilustrativos:

```
import itertools

for numero in itertools.count(20, 2):
    print(numero)
```

³ Disponível em: <<https://docs.python.org/3/library/itertools.html>>. Acesso em: 31 jan. 2022.

⁴ Disponível em: <<https://docs.python.org/3/library/itertools.html#itertools.count>>. Acesso em: 31 jan. 2022.

⁵ Disponível em: <<https://docs.python.org/3/library/itertools.html#itertools.cycle>>. Acesso em: 31 jan. 2022.

⁶ Disponível em: <<https://docs.python.org/3/library/itertools.html#itertools.repeat>>. Acesso em: 31 jan. 2022.



```
20
22
24
26
28
30
...
```

Imagine o *cycle* como um carrossel: ele percorrerá todos os valores de uma lista (ou *string*, ou *array*, ou alguma outra variável iterável) até o seu final. Ao terminar, voltará a mostrar os valores no começo daquele *array* e assim indefinidamente. Novamente, para fins de ilustração, somente as primeiras entradas são mostradas. Observe que realçamos em amarelo o primeiro elemento da lista para facilitar a sua visualização dos resultados.

```
for valor in itertools.cycle(lista_equipas):
    print(valor)
```

```
Athletic 220
Falcons
Ulaanbaatar
Khaan Khuns - Erchim
Deren
Ulaanbaatar City
Khangarid
Khoromkhon
BCH Lions
UB Mazaalaynuud
Athletic 220
Falcons
Ulaanbaatar
Khaan Khuns - Erchim
Deren
Ulaanbaatar City
Khangarid
Khoromkhon
BCH Lions
UB Mazaalaynuud
Athletic 220
Falcons
Ulaanbaatar
...
```

Já o *repeat* retorna o mesmo objeto que você forneceu repetidas vezes. Você pode opcionalmente informar quantas repetições deseja: no exemplo a seguir, escolhemos três. Se não informássemos nada, este objeto (no exemplo, a lista) seria repetido infinitas vezes.

```
for valor in itertools.repeat(lista_equipas, 3):
    print(valor)
```



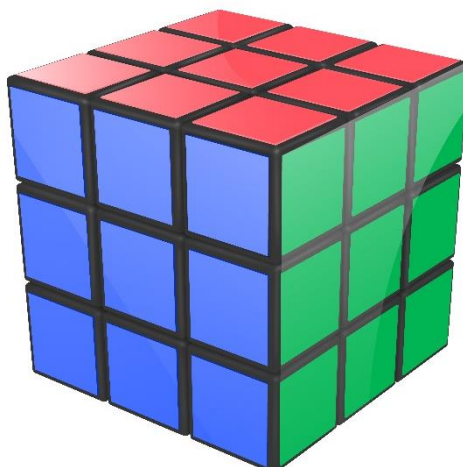
```
['Athletic 220' 'Falcons' 'Ulaanbaatar' 'Khaan Khuns - Erchim' 'Deren'  
'Ulaanbaatar City' 'Khangarid' 'Khoromkhon' 'BCH Lions' 'UB Mazaalaynuud']  
['Athletic 220' 'Falcons' 'Ulaanbaatar' 'Khaan Khuns - Erchim' 'Deren'  
'Ulaanbaatar City' 'Khangarid' 'Khoromkhon' 'BCH Lions' 'UB Mazaalaynuud']  
['Athletic 220' 'Falcons' 'Ulaanbaatar' 'Khaan Khuns - Erchim' 'Deren'  
'Ulaanbaatar City' 'Khangarid' 'Khoromkhon' 'BCH Lions' 'UB Mazaalaynuud']
```

TEMA 2 – COMBINATÓRIOS

Vamos pausar um pouco as coisas sob um ponto de vista de código e pensar sob um ponto de vista de problemas. Digamos que você não queira somente **percorrer** a lista (iterar por ela), mas sim gerar **ordens diferentes** dessa lista e, aí, usá-la. Vejamos alguns casos nos quais isso pode ser útil do ponto de vista de programação:

- Gerar ordenações diferentes de um baralho de cartas para um jogo.
- Gerar uma amostra válida de placas de carro.
- Gerar uma ordem diferente de respostas para um sistema *online* de provas.
- Obter uma amostra de casos a serem analisados para um trabalho de análise de dados.
- Criar combinações de *playlists* de músicas.
- Obter uma amostra de pessoas para um comitê escolhido por um algoritmo.
- Descobrir alternativas para fazer várias entregas de moto ou caminhão.

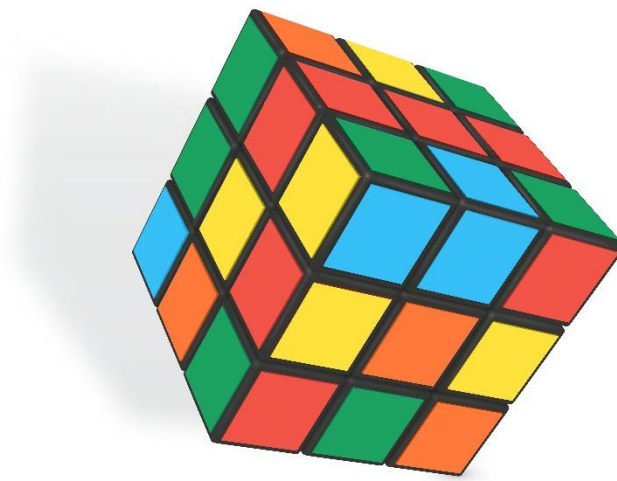
Imaginemos agora um exemplo mais visual: pensemos em um cubo mágico.



Créditos: Makstorm/Shutterstock.

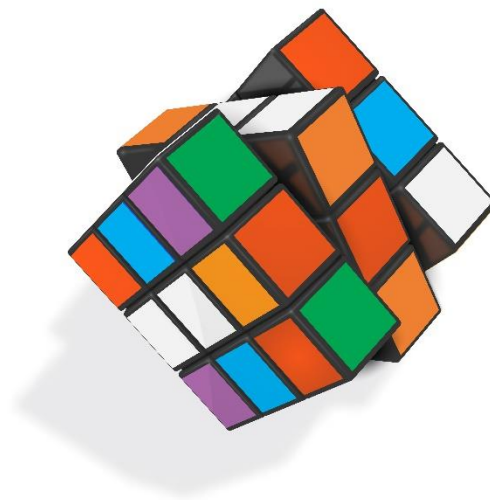


Este mesmo cubo poderia ser organizado assim:



Créditos: Makstorm/Shutterstock.

Ou até mesmo poderia ficar próximo disso:



Créditos: Makstorm/Shutterstock.

O ponto é: que outras configurações poderiam ser feitas? Que outras configurações válidas teríamos? Mudemos o escopo da pergunta: que outras combinações **válidas** de CPF ou CNPJ existem? Ou, que outras combinações válidas de códigos de boleto, cartão de crédito e afins poderíamos ter? Observe que a teoria por trás de todas essas perguntas é bem similar e, ainda, possui um escopo bem atual em relação ao nosso trabalho em TI: independentemente da área da indústria na qual possamos atuar (isto é, manufatura, logística, entretenimento, esportes, serviços, telecomunicações, entre outros), essa



temática de fato possui uma aplicação real. Entendemos que é importante destacarmos isso, uma vez que é um pouco mais difícil enxergarmos uma proximidade desse tema com o universo de TI em comparação com as outras temáticas.

2.1 Combinações

As **combinações** geram... bom, combinações com base em um conjunto predeterminado de elementos. Imagine esse conjunto como um grande grupo de elementos (ou possibilidades) do qual podemos escolher o que importa para nós para uma determinada ocasião.

Pensem no seguinte problema: você resolve colocar em prática os seus conhecimentos para criar um algoritmo que gere uma combinação de roupas que você poderá usar durante a semana. É, para todos os efeitos, um código para **planejar** as suas roupas para os próximos dias. Começemos com as opções da parte de cima do corpo. Neste caso, você hoje possui as seguintes opções disponíveis:

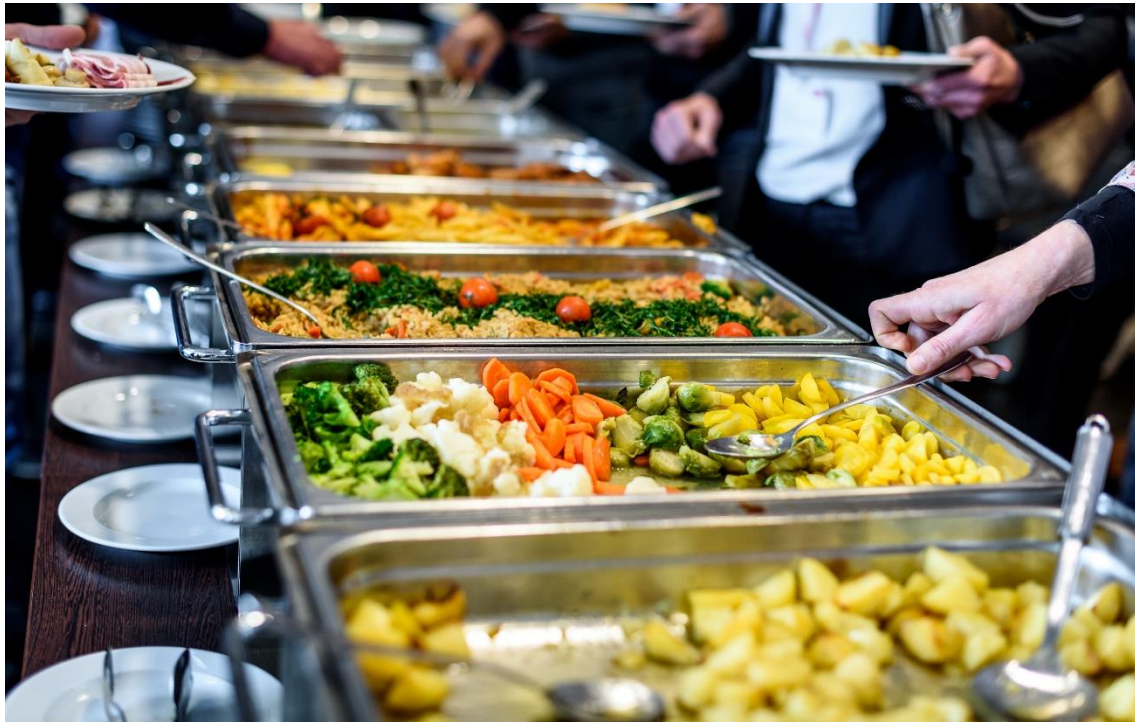
- Camiseta preta.
- Camisa branca.
- Regata branca.
- Camiseta amarela.
- Camiseta cinza.
- Camisa roxa.
- Camisa azul.
- Camisa verde.
- Regata preta.
- Camisa xadrez.

Vamos supor que queiramos extrair (ou obter uma amostra) de uma roupa, somente. Ora, é fácil: podemos escolher **qualquer** um dos itens – *camiseta preta* é uma resposta válida, e *regata preta*, também, ou *camisa xadrez*. *Regata rosa* não é, porque ela não está na lista.

Agora, digamos que queremos dois itens: a mesma lógica persiste – *camiseta preta* e *camisa branca* é uma resposta válida, bem como *camiseta branca* e *camiseta preta* ou *camisa verde* e *regata branca*, e assim por diante. Por outro lado, *camiseta branca* e *regata rosa* não é uma combinação possível

porque, ainda que a camiseta branca esteja na lista original, a regata rosa não está. Bom, você pegou o jeito, certo?

Figura 1 – Este é um buffet. Dependendo da parte do país na qual você mora isto pode ter outro nome. Por isso é importante mostrarmos um exemplo do que falamos



Créditos: JGA/Shutterstock.

Quando trabalhamos com combinações, existem alguns parâmetros a serem observados:

- Um **iterável** (por exemplo, uma lista ou uma *string*) o qual possui os itens que podemos utilizar: imagine isso como um *buffet* em que podemos escolher qualquer coisa para colocar no nosso prato **desde que faça parte do buffet**.
- A **quantidade** de elementos que precisamos selecionar como resultado: queremos todas as combinações possíveis com 2 elementos (como as 2 peças de roupas que vimos acima)? Ou 3 elementos? Ou 5 elementos?
- Se **repetições do mesmo item** são aceitáveis ou não: podemos reaproveitar uma mesma camisa 2 vezes? Ou, no caso do *buffet*, podemos colocar arroz 3 vezes no prato e mais nada?

Vamos pensar em como isso funcionaria em Python. Para começo de conversa, retomaremos a lista de roupas:



```
lista_roupas = ['Camiseta preta', 'Camisa branca', 'Regata branca',  
                'Camiseta amarela', 'Camiseta cinza', 'Camisa roxa',  
                'Camisa azul', 'Camisa verde', 'Regata preta',  
                'Camisa xadrez']
```

Com esta lista em mãos, vamos utilizar as funções *combinations*⁷ e *combinations_with_replacement*⁸ do módulo *itertools*⁹, módulo este que já é parte da biblioteca-padrão do Python. O código a seguir mostra a quantidade de combinações que existem com base nas dez peças citadas. Por exemplo, existem 45 combinações possíveis de 2 peças **com base nas 10 peças** que listamos em *lista_roupas* e 252 combinações possíveis de 5 peças **observando as mesmas 10 peças**.

```
from itertools import combinations  
  
# obtendo a quantidade de combinações por quantidade de peças  
for i in range(1, len(lista_roupas)):  
    n_combinacoes = len(list(combinations(lista_roupas, i)))  
    print(f'Existem {n_combinacoes} combinações com {i} peças.')
```

```
Existem 10 combinações com 1 peças.  
Existem 45 combinações com 2 peças.  
Existem 120 combinações com 3 peças.  
Existem 210 combinações com 4 peças.  
Existem 252 combinações com 5 peças.  
Existem 210 combinações com 6 peças.  
Existem 120 combinações com 7 peças.  
Existem 45 combinações com 8 peças.  
Existem 10 combinações com 9 peças.
```

Vamos ver quais combinações são geradas com duas roupas, por exemplo. No caso, algumas delas, dada a quantidade de combinações possíveis:

⁷ Disponível em: <<https://docs.python.org/3/library/itertools.html#itertools.combinations>>. Acesso em: 31 jan. 2022.

⁸ Disponível em: <https://docs.python.org/3/library/itertools.html#itertools.combinations_with_replacement>. Acesso em: 31 jan. 2022.

⁹ Disponível em: <<https://docs.python.org/3/library/itertools.html>>. Acesso em: 31 jan. 2022.



```
# mostrando todas as combinações possíveis com duas roupas
for combinacao in combinations(lista_roupas, 2):
    print(combinacao)
```

```
('Camiseta preta', 'Camisa branca')
('Camiseta preta', 'Regata branca')
('Camiseta preta', 'Camiseta amarela')
('Camiseta preta', 'Camiseta cinza')
('Camiseta preta', 'Camisa roxa')
('Camiseta preta', 'Camisa azul')
('Camiseta preta', 'Camisa verde')
('Camiseta preta', 'Regata preta')
('Camiseta preta', 'Camisa xadrez')
('Camisa branca', 'Regata branca')
('Camisa branca', 'Camiseta amarela')
('Camisa branca', 'Camiseta cinza')
('Camisa branca', 'Camisa roxa')
('Camisa branca', 'Camisa azul')
('Camisa branca', 'Camisa verde')
('Camisa branca', 'Regata preta')
('Camisa branca', 'Camisa xadrez')
('Regata branca', 'Camiseta amarela')
('Regata branca', 'Camiseta cinza')
...
```

Veja que aparecem todas as combinações possíveis envolvendo a camiseta preta (que era o primeiro item da nossa lista), seguidas pelas combinações possíveis da camisa branca (que era o segundo item), seguidas pelas combinações da regata branca (que era o terceiro item), e assim sucessivamente. Veja que o par camiseta preta e camisa branca só aparece uma vez, ou seja, este par não se repete porque se entende que queremos vê-lo partindo da premissa de que camiseta preta e camisa branca são a mesma coisa de camisa branca e camiseta preta, logo a ordem dos dois não importa e, por isso, somente não existem duplicatas neste sentido. Agora, o que acontece se trocarmos o *combinations* pelo *combinations_with_replacement*?

```
from itertools import combinations_with_replacement

# obtendo a quantidade de combinações por quantidade de peças
for i in range(1, len(lista_roupas)):
    n_combinacoes = len(list(combinations_with_replacement(lista_roupas, i)))
    print(f'Existem {n_combinacoes} combinações com {i} peças.')
```




Existem 10 combinações com 1 peças.
Existem 55 combinações com 2 peças.
Existem 220 combinações com 3 peças.
Existem 715 combinações com 4 peças.
Existem 2002 combinações com 5 peças.
Existem 5005 combinações com 6 peças.
Existem 11440 combinações com 7 peças.
Existem 24310 combinações com 8 peças.
Existem 48620 combinações com 9 peças.

O primeiro ponto para observarmos é a quantidade de combinações: se antes o número de combinações alcançava um pico com 5 peças, agora a quantidade só aumenta. Com 9 peças, havia 10 combinações, e, agora, existem 48.620. Vejamos alguns exemplos de combinações de duas peças com esta função:

```
# mostrando todas as combinações possíveis com duas roupas
for combinacao in combinations_with_replacement(lista_roupas, 2):
    print(combinacao)
```

```
('Camiseta preta', 'Camiseta preta')
('Camiseta preta', 'Camisa branca')
('Camiseta preta', 'Regata branca')
('Camiseta preta', 'Camiseta amarela')
('Camiseta preta', 'Camiseta cinza')
('Camiseta preta', 'Camisa roxa')
('Camiseta preta', 'Camisa azul')
('Camiseta preta', 'Camisa verde')
('Camiseta preta', 'Regata preta')
('Camiseta preta', 'Camisa xadrez')
('Camisa branca', 'Camisa branca')
('Camisa branca', 'Regata branca')
('Camisa branca', 'Camiseta amarela')
('Camisa branca', 'Camiseta cinza')
('Camisa branca', 'Camisa roxa')
('Camisa branca', 'Camisa azul')
('Camisa branca', 'Camisa verde')
('Camisa branca', 'Regata preta')
('Camisa branca', 'Camisa xadrez')
('Regata branca', 'Regata branca')
('Regata branca', 'Camiseta amarela')
('Regata branca', 'Camiseta cinza')
...
```

Veja os novos casos em amarelo: neste contexto, o `combinations_with_replacement` permite que sejam geradas combinações em que um elemento pode se repetir. É por essa razão que temos muito mais combinações desta forma. Na analogia das roupas, o `combinations` não permite que usemos a mesma roupa duas ou mais vezes; já o `combinations_with_replacement` permite.



TEMA 3 – PERMUTAÇÕES

As **permutações** possuem uma relação muito próxima com a **ordenação**, isto é, quais são as combinações possíveis **quando a ordenação entre eles importa?**

Os pré-requisitos que comentamos anteriormente são praticamente os mesmos:

- Continuamos precisando de um **iterável**.
- A **quantidade** de elementos que precisamos selecionar como resultado: ao contrário das combinações, a **ordem** importa.
 - Uma combinação válida é camisa preta e camisa branca: são duas camisas separadas e não importa a ordem na qual você utilizará – é uma combinação válida, e é isso que importa para as combinações.
 - Já duas permutações válidas são camisa preta e camisa branca e camisa branca e camisa preta. Veja que aqui são **dois** resultados diferentes. Nas combinações, não teríamos esses dois casos separados, mas somente um resultado. Percebe que a ordem passa a importar aqui?
 - Pensemos novamente em analogias: para algumas pessoas, não importa se o arroz vai em cima do feijão ou se o feijão vai em cima de arroz. Para esses casos, teríamos um cenário de **combinação**. Para outras, a ordem importa muito – aí é um caso de **permutação**.

Retornemos ao exemplo da lista de roupas que utilizamos nos combinatórios: a lógica é bem parecida para gerarmos as permutações. O `itertools` no Python também oferece uma função neste sentido: a *permutations*¹⁰.

```
from itertools import permutations

# obtendo a quantidade de permutações por quantidade de peças
for i in range(1, len(lista_roupas)):
    n_permutacoes = len(list(permutations(lista_roupas, i)))
    print(f'Existem {n_permutacoes} permutações com {i} peças.')
```

¹⁰ Disponível em: <<https://docs.python.org/3/library/itertools.html#itertools.permutations>>. Acesso em: 31 jan. 2022.



Existem 10 permutações com 1 peças.
Existem 90 permutações com 2 peças.
Existem 720 permutações com 3 peças.
Existem 5040 permutações com 4 peças.
Existem 30240 permutações com 5 peças.
Existem 151200 permutações com 6 peças.
Existem 604800 permutações com 7 peças.
Existem 1814400 permutações com 8 peças.
Existem 3628800 permutações com 9 peças.

A quantidade de permutações geradas foi ainda maior do que com `combinations_with_replacement`. Como é de se esperar, há um porquê: lembra que na permutação a ordem dos elementos é importante? Observemos alguns exemplos de pares de roupas gerados:

```
# mostrando todas as permutações possíveis com duas roupas
for permutacao in permutations(lista_roupas, 2):
    print(permutacao)
```

```
('Camiseta preta', 'Camisa branca')
('Camiseta preta', 'Regata branca')
('Camiseta preta', 'Camiseta amarela')
('Camiseta preta', 'Camiseta cinza')
('Camiseta preta', 'Camisa roxa')
('Camiseta preta', 'Camisa azul')
('Camiseta preta', 'Camisa verde')
('Camiseta preta', 'Regata preta')
('Camiseta preta', 'Camisa xadrez')
('Camisa branca', 'Camiseta preta')
('Camisa branca', 'Regata branca')
('Camisa branca', 'Camiseta amarela')
('Camisa branca', 'Camiseta cinza')
('Camisa branca', 'Camisa roxa')
('Camisa branca', 'Camisa azul')
('Camisa branca', 'Camisa verde')
('Camisa branca', 'Regata preta')
('Camisa branca', 'Camisa xadrez')
('Regata branca', 'Camiseta preta')
('Regata branca', 'Camisa branca')
('Regata branca', 'Camiseta amarela')
('Regata branca', 'Camiseta cinza')
...
```

Observemos os casos realçados: em verde, temos dois pares em que era somente um para as combinações. Ou seja, enquanto para as combinações a ordem não importava e, por isso, tínhamos apenas um par representando a junção de camiseta preta e camisa branca, agora é importante a ordem, e, por isto, temos ambos os casos respeitando a mesma ordem. Isso também vale para o caso em azul (entre a camiseta preta e a regata branca), bem como para todas as outras peças de roupa da lista original.



TEMA 4 – GENERATORS

Generators são, de forma sucinta, funções que retornam (e funcionam como) iteradores. Assim, você pode criar os seus próprios iteradores. Agora, uma das vantagens dos *generators* é a possibilidade de uso de *lazy iterators*: a palavra *lazy* aqui indica que o cálculo ou a determinação dos resultados são atrasados o máximo possível (ou seja, até que realmente precisemos daquele resultado). Isto é bem útil quando começamos a trabalhar com *big data* ou qualquer outro cenário em que temos relativamente pouca memória disponível. Nestes casos, é possível manipularmos as variáveis e determinarmos os resultados mesmo que o volume de dados seja bem maior que aquele que temos disponível para uso.

Vamos relembrar as funções que já trabalhamos anteriormente em diversas oportunidades: as funções geralmente podem receber parâmetros e, além disso, podem (ou não) retornar valores. De forma bem simplista, imaginemos que os *generators* retornarão um iterador em vez de um valor, é isso.

Além disso, os *generators* (ao menos os que são feitos em Python) não usam o *return*, mas o *yield*. A palavra-chave *yield* poderia ser entendida como “produza isso, mas não termine ainda de encerrar essa função”. Essa é uma diferença bem importante em relação ao *return*:

- O *return* terminará a função e retornará os valores produzidos até o momento. É como se fôssemos a um supermercado, pegássemos todos os produtos que quiséssemos e fôssemos embora logo em seguida.
- Já o *yield* retornará um valor de cada vez. É como se estivéssemos no final de uma linha de produção de uma fábrica e fôssemos coletando um produto que fosse sendo produzido de cada vez.

A analogia pode funcionar bem aqui para entendermos a importância e a diferença entre ambos. Imagine que uma pessoa tenha somente as mãos para pegar alguns produtos – ou seja, ela não possui um carro, sacolas ou qualquer outra coisa que a ajude a carregar os produtos.



Créditos: CKP1001/Shutterstock.

Como comentamos anteriormente, esse carrinho seria o *return*: é fácil de levar o que tem aqui usando somente as mãos **se e somente se** a quantidade de itens for baixa. Já com o carrinho cheio, assim seria impossível. No mundo do desenvolvimento de algoritmos, imagine essa analogia como sendo um *return* contendo um alto volume de dados quando não temos memória suficiente para lidar com esse volume.



Créditos: hedgehog94/Shutterstock.

Por outro lado, a linha de produção seria o *yield*: ainda que a linha de produção possa não ter fim (veja que existem várias garrafas, uma atrás da outra



e aparentemente sem fim), conseguimos facilmente pegar uma de cada vez com as mãos. Voltando ao mundo do desenvolvimento de algoritmos, o uso do *yield* permite um baixo uso de memória, então não precisaríamos esperar o fim do processamento: podemos já manipular os valores que vão sendo criados pelo *generator*.

Vejamos um exemplo na prática inspirado nos exemplos da Wiki¹¹ oficial do Python. Vamos supor que queremos gerar uma lista de recibos, a qual é uma sequência de códigos começando com “Recibo 1”, passando para “Recibo 2”, “Recibo 3” e assim sucessivamente até alcançarmos um valor limite.

```
def gerar_recibos(limite):
    contador = 1
    recibos = []

    while contador < limite:
        recibos.append(f'Recibo {contador}')
        contador += 1

    # após criar os recibos retornamo-los
    return recibos

# a linha abaixo dará um estouro de memória em vários computadores
lista_recibos = gerar_recibos(100000000000)
```

O código possui uma função que gera os recibos e retorna-os em uma lista. O problema é quando a quantidade de recibos é muito alta (como o número visto): o código ficará rodando por vários segundos até que a memória estoure em algum momento, e, com isso, não teremos nada em mãos. Veja como ficaria a mesma lógica no código a seguir com o uso do *generator*: some o *return* e a lista de recibos na função, e entra o *yield* em cena. O código a seguir é executado em instantes:

```
def gerar_recibos_generator(limite):
    contador = 1

    while contador < limite:
        yield f'Recibo {contador}'
        contador += 1

lista_recibos = gerar_recibos_generator(100000000000)
```

¹¹ Disponível em: <<https://wiki.python.org/moin/Generators>>. Acesso em: 31 jan. 2022.



O `lista_recibos` não é mais uma lista, mas um objeto do *generator*. Isso significa que os valores em si ainda não foram gerados, mas estão prontos para serem gerados um de cada vez dependendo do uso. Por exemplo: para retornar os dez primeiros valores, temos o código a seguir, o qual roda em instantes e sem sobrecarregar a memória (já que só precisamos desses dez primeiros – logo, os demais jamais são criados):

```
count = 0
for recibo in lista_recibos:
    display(recibo)

    # incrementando o contador
    count += 1

    # parando de mostrar os recibos se já mostramos 10
    if count >= 10:
        break
```

```
'Recibo 1'
'Recibo 2'
'Recibo 3'
'Recibo 4'
'Recibo 5'
'Recibo 6'
'Recibo 7'
'Recibo 8'
'Recibo 9'
'Recibo 10'
```

TEMA 5 – SOBRECARGA DE OPERADORES

Vamos simplificar este conceito o dividindo em dois tópicos: *operadores* e *sobrecarga*.

Quando estamos falando de *operadores*, estamos falando dos símbolos¹² que são utilizados para fazer alguma operação. Por padrão, temos alguns exemplos:

- “+” é utilizado para somar.
- “-” é utilizado para subtrair.
- “/” é utilizado para dividir.
- “%” é utilizado para obter o resto de uma divisão.

¹² Disponível em: <<https://docs.python.org/3/library/operator.html#mapping-operators-to-functions>>. Acesso em: 31jan. 2022.



Ou seja, até este momento, você certamente já trabalhou com vários operadores e sabe como aplicá-los corretamente em Python. Agora, o que seria uma sobrecarga? Entende-se como sobrecarga um caso no qual um operador pode ter novos comportamentos e implementações do que ele deveria significar dependendo dos atributos associados a ele.

Ao observarmos essa definição de sobrecarga, é provável que você tenha associado com algo que vimos anteriormente: **polimorfismo**. Na realidade, o que havíamos visto naquele momento era uma **sobrecarga de função**. Logo, esta **sobrecarga de operadores** que vemos agora nada mais é do que outro tipo de polimorfismo.

Bom, e como funciona a sobrecarga de operadores em Python? Começemos com o nome de dois cachorros em duas *strings* separadas:

```
print('Pippoca' + 'Kora')  
print('Pippoca' * 3)
```

```
PippocaKora  
PippocaPippocaPippoca
```

Quando pensamos no contexto de fora do desenvolvimento de algoritmo, não faria muito sentido somar textos, mas aqui em Python há uma sobrecarga implementada para *strings* de forma que o sinal de “+” equivale à **concatenação** de *strings*: daí o resultado “PippocaKora”. Já a “*” segue o mesmo contexto: a implementação deste operador em Python implica que a *string* é **repetida** algumas vezes: três vezes, no nosso caso, resultando em “PippocaPippocaPippoca”. Isso também vale para listas: o “+” concatena duas listas, e o “*” repete os elementos da lista em uma nova lista maior:

```
print(['Pippoca', 'Kora'] + ['Dexter', 'Pulga'])  
print(['Pippoca', 'Kora'] * 4)
```

```
['Pippoca', 'Kora', 'Dexter', 'Pulga']  
['Pippoca', 'Kora', 'Pippoca', 'Kora', 'Pippoca', 'Kora', 'Pippoca', 'Kora']
```

Agora, pensemos em uma classe chamada *Cachorro* (lembra do conceito de classe que vimos anteriormente?). Esta classe *Cachorro* possui dois atributos, somente: nome e idade. Criaremos dois objetos com base nessa classe: um que representa um cachorro chamado *Pippoca* e outro que representa a chamada *Kora*:



```
class Cachorro():
    def __init__(self, nome=None, idade=0):
        self.__nome = nome
        self.__idade = idade

    def ler_idade(self):
        return self.__idade

pippoca = Cachorro('Pippoca', 9)
kora = Cachorro('Kora', 1)
```

Beleza, mas o que acontece se eu tento **somar** os dois?

```
pippoca + kora
```

```
TypeError                                Traceback (most recent call last)
in <module>()
----> 1 pippoca + kora

TypeError: unsupported operand type(s) for +: 'Cachorro' and 'Cachorro'
```

O erro realçado em amarelo nos informa que basicamente não há em lugar algum uma especificação sobre o que se deve fazer com o sinal de “+” para a classe Cachorro – ou seja, não esclarecemos em nenhum lugar o que se entende por “soma de Cachorro”. E isto é verdade: o que devemos fazer? Essa frase faz sentido, em primeiro lugar? O que seria uma “soma de Cachorro”? Somaríamos as idades? Os nomes? Criaríamos um cachorro com oito patas e duas cabeças? Ou inventaríamos um cachorro de dois andares? O que você entende por soma de Cachorro?

Se temos uma noção clara do que queremos fazer, é possível implementarmos isso em Python, mas somos nós que devemos fazê-lo, e não esperar que a execução do algoritmo adivinhe isso para nós. Na documentação oficial do Python, temos a lista de métodos¹³, que podemos utilizar para fazer essa sobrecarga: para o “+” seria o “__add__”, por exemplo. Logo, vamos modificar a classe Cachorro para que a soma dela signifique na soma das **idades** dos cachorros. Logo:

¹³ Disponível em: <<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>. Acesso em: 31 jan. 2022.



```
class Cachorro():
    def __init__(self, nome=None, idade=0):
        self.__nome = nome
        self.__idade = idade

    def ler_idade(self):
        return self.__idade

    # implementando a sobrecarga de operadores
    def __add__(self, outro_cachorro):
        return self.__idade + outro_cachorro.__idade

pippoca = Cachorro('Pippoca', 9)
kora = Cachorro('Kora', 1)

pippoca + kora
```

10

Observe que agora a soma funcionou adequadamente, retornando a soma da idade dos dois cachorros (no caso, 9 anos + 1 ano = 10 anos). Poderíamos também implementar funções para subtração, divisão, multiplicação e outros seguindo a mesma fórmula.

FINALIZANDO

Nesta aula, tivemos como foco compreender alguns tópicos avançados em desenvolvimento. Como já comentamos antes, esses tópicos foram exemplificados em Python, mas não são exclusivos para o Python. Logo, esses mesmos conceitos se aplicam para outras linguagens de programação.

O uso de iteradores, combinatórios e geradores auxilia no uso sensato de recursos computacionais (como memória e processador) para construirmos *softwares* que tenham um consumo consciente de recursos. Imagine uma solução construída para uma grande indústria e que seja executada em alguma arquitetura na nuvem: uma pequena redução no consumo de memória pode implicar na economia de alguns milhares de reais por mês, por exemplo. Ainda, conseguimos utilizar funções já implementadas que coloquem em prática esta lógica em vez de investirmos muitas linhas de código para reescrever algo complexo e que requerirá uma manutenção adicional posteriormente: dizem que uma das virtudes de um bom desenvolvedor é a preguiça – o bom desenvolvedor



fará um bom código logo de cara para reduzir o esforço em dar suporte ao código e ficar corrigindo *bugs* depois, para reduzir a quantidade de dúvidas sobre o código e para não ser incomodado fora das suas horas de trabalho. Seguindo essas premissas, conhecer técnicas que possam reduzir o nosso esforço ajudam muito a atingir esses objetivos.



REFERÊNCIAS

AGUILAR, L. J. **Fundamentos de programação**: algoritmos, estruturas de dados e objetos. 3. ed. Porto Alegre: AMGH, 2011.

GRUS, J. **Data Science do zero**. Rio de Janeiro: Alta Books, 2021.

PERKOVIC, L. **Introdução à computação usando Python** - um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.